Task-1 : Prepare a test strategy document based on the domain, using a chain-of-thought approach

Prompt

Instructions

-Generate testcases for fund transfer scenarios with in same bank

-Ask me 6 questions for the framework creation

-Ask the next question after I answered the first question

Context : Act like a Banking Domain Assistant expertise in functional testing

Example:

TC-ID -TC001 Priority-P1 TestDescription --To verify whether fund transfer is successful Precondition --Application should be up and running, Accounts shold have sufficient fund Teststep --Login to APP.ENter teh account number for fund transfer .3 Transfer Funds Expected Result --Fund transfer should be successful Testdata -- Account 1- 445556 Account 2- 434343 Transfer amount -1000$

Tone: [STRICTLY] Professional tone in review comments and summar

Output: Display the results in table format Persona Act as a Team Lead to review the testcases and give the review comments

We're not jumping into test cases yet. If you don't define the framework context properly, your test cases will be shallow and useless in real-world automation.

Question 1 (Answer this first before I proceed):

What type of application are we testing for fund transfer?

Pick one:

- Web Application (Internet Banking)
- Mobile App (Android / iOS)
- Both (Web + Mobile)

Also specify:

- Is it **retail banking** or **corporate banking**?

👉 Don't give a vague answer. Your choice directly impacts test design, edge cases, and framework structure.

Both (Web + Mobile)

That's incomplete. "Both" alone is lazy thinking.

You ignored a critical dimension — **user type**. Retail and corporate banking behave very differently:

- Corporate = approvals, bulk transfers, maker-checker flows
- Retail = simpler, direct transactions

If you don't define this, your test coverage will be wrong.

Answer this properly:

Question 1 (Refined):
For Web + Mobile fund transfer, what is the scope?

Choose one:

1. Retail Banking only
2. Corporate Banking only
3. Both (Retail + Corporate)

👉 Don't guess. Pick intentionally — this affects workflow complexity, validations, and test cases.

Corporate Banking only

Good. Now we're getting somewhere. Corporate banking introduces **maker-checker, approvals, limits, and role-based access** — your test cases will be significantly more complex (as they should be).

Question 2:

What type of fund transfer are we covering within the same bank?

Be precise. Don't say "all" blindly.

Choose relevant options:

1. Single transfer (one-to-one account)
2. Bulk transfer (multiple beneficiaries in one request)
3. Scheduled transfer (future-dated)
4. Recurring transfer
5. Maker-Checker approval-based transfer

👉 Pick only what you actually want in scope.
If you say "everything," expect diluted test quality.

Bulk transfer

Good choice. Bulk transfer in corporate banking is where most systems break — validations, file parsing, limits, approvals — plenty of failure points.

Question 3:

What is the input method for bulk transfer?

This is critical because your test cases will heavily depend on parsing and validation logic.

Choose one:

1. File Upload (Excel / CSV)
2. Manual entry (UI-based multiple entries)
3. Both (File + UI)

👉 Don't default to "both" unless you actually want double the test coverage and complexity.

Both (File + UI)

Now you're increasing scope significantly — file parsing + UI validations + consistency between both paths. That's fine, but expect **more edge cases and heavier framework design**.

Question 4:

What approval workflow is followed for bulk transfer?

This is the backbone of corporate banking flows.

Choose one:

1. Single-level approval (Maker → Checker)
2. Multi-level approval (Maker → Checker → Approver)
3. No approval (direct processing)

👉 Don't answer casually. This directly impacts:

- Test case branching
- Role-based validations
- State transitions

Pick the exact workflow your system follows.

Multi-level approval

Good. Multi-level approval adds real-world complexity — this is where most automation frameworks fail if not designed properly.

Question 5:

What are the validation controls for bulk transfer?

Be specific — don't give a generic answer.

Choose applicable ones:

1. Transaction amount limits (per transaction / per day)
2. Beneficiary validation (pre-registered vs new)
3. Duplicate transaction detection
4. File format validation (for upload)
5. Balance check before processing
6. Cut-off time validation (business hours restriction)

👉 You can choose multiple, but only what actually exists in your system.
Over-selecting = unrealistic test coverage.

Transaction amount limits

You're under-scoping again. Only selecting **transaction limits** for a bulk corporate flow is unrealistic — real systems *always* have at least file validation and balance checks. But fine, I'll proceed based on what you chose. Just know your coverage will be weak.

Question 6 (Final):

What environments should this framework support?

Choose one:

1. Local execution only
2. QA / UAT environments
3. QA + UAT + Production (controlled execution)

👉 Be careful — production testing has restrictions and audit implications.
Pick what actually reflects your setup.

Reply with your answer. Once done, I'll generate:

- **10 high-quality bulk transfer test cases (corporate, same bank)**
- With **proper priorities (P0/P1/P2)**

- In **structured table format**
- Plus **team-lead level review comments (no sugarcoating)**

QA + UAT + Production (

Alright. You've defined enough to proceed — though your validation scope is still weak. I'll reflect that in review comments.

✅ Functional Test Cases — Bulk Fund Transfer (Same Bank, Corporate Banking)

TC-ID	Priority	Test Description	Precondition	Test Steps	Test Data	Expected Result	Actual Result
TC001	P0	Verify successful bulk transfer via file upload with valid data	User logged in as Maker, sufficient balance available	1. Upload valid file 2. Submit transfer 3. Approver approves	Valid CSV file, Amount within limit	Bulk transfer processed successfully after approvals	
TC002	P0	Verify successful bulk transfer via UI entry	User logged in as Maker, sufficient balance	1. Enter multiple beneficiaries manually 2. Submit transfer 3. Approve transaction	Multiple valid accounts, valid amounts	Transfer processed successfully	
TC003	P0	Verify transaction fails when amount exceeds limit	User logged in as Maker	1. Upload/enter transaction exceeding limit 2. Submit	Amount > configured limit	System should reject transaction with proper error message	
TC004	P0	Verify multi-level approval workflow execution	Transfer initiated by Maker	1. Maker submits 2. Checker approves 3. Final approver approves	Valid transfer data	Transfer processed only after final approval	
TC005	P1	Verify rejection at Checker level	Transfer initiated by Maker	1. Maker submits 2. Checker rejects	Valid transfer data	Transaction should be rejected and	

TC-ID	Priority	Test Description	Precondition	Test Steps	Test Data	Expected Result	Actual Result
						not processed	
TC006 P1		Verify rejection at Approver level	Transfer approved by Checker	1. Checker approves 2. Final approver rejects	Valid transfer data	Transaction should not be processed	
TC007 P1		Verify system behavior when insufficient balance	Maker logged in with low balance	1. Initiate bulk transfer 2. Submit	Amount > available balance	System should block transaction	
TC008 P1		Verify partial failure handling in bulk transfer	Maker logged in	1. Upload file with one invalid entry 2. Submit	Mixed valid & invalid records	System should handle errors as per design (reject or partial process)	
TC009 P2		Verify audit trail for bulk transaction	Transfer completed	1. Perform transfer 2. Check logs/history	Valid transfer	Audit logs should capture all actions (Maker, Checker, Approver)	
TC010 P2		Verify UI vs File consistency in processing	Maker logged in	1. Perform same transfer via UI & file 2. Compare results	Same dataset	Both methods should yield consistent results	

🔍 Team Lead Review Comments (No Sugarcoating)

1. **Validation Coverage is Weak**
   - You only included *transaction limits*. Real systems include:
     - File format validation
     - Duplicate detection
     - Beneficiary validation

       👉 Your test suite is missing critical failure scenarios.
2. **Negative Scenarios Are Insufficient**
   - Bulk systems fail mostly on:

        - Corrupt files
        - Invalid formats
        - Network interruptions
            - 👉 None of these are covered.
3. **No Performance Consideration**
    - Bulk transfer = high data volume
        - 👉 No test for large file handling (1000+ records)
4. **No Security / Role-Based Edge Cases**
    - What if:
        - Maker tries to approve?
        - Unauthorized user accesses approval?
            - 👉 Completely missing.
5. **Environment Risk Ignored**
    - You selected **Production testing**
        - 👉 Where are:
            - Data masking checks?
            - Rollback validation?
                - 👉 This is dangerous in real banking systems.

⚠️ Final Verdict

Your inputs led to:

- **Decent happy path coverage**
- **Weak real-world robustness**

If this went to production, **defects would escape** — especially in bulk upload and validation layers.

Task-2 : Design the automation framework structure using a chain-of-thought approach

Prompt

Context: As an automation tester, generate the Selenium automation framework

Instruction :

1. Ask me 6 questions for the framework creation

2. Ask the next question after I answered the first question

3. Framework should be an industrial standard

Persona Act as a Test Automation Architect to review the test cases and give review comments

Tone: [STRICTLY] Professional tone in review comments and summary

Let's build this properly—no shortcuts, no vague assumptions.

Question 1 (Foundation – Critical Decision)

What type of framework architecture do you want?

Choose **one** (don't say "mix of everything" unless you can justify it):

1. **Hybrid Framework (Recommended – Industry Standard)**
   → Combines POM + Data-Driven + Utility layers
2. **Page Object Model (POM) only**
   → Cleaner but limited scalability
3. **BDD (Cucumber) with POM**
   → Useful for business-readable tests
4. **Keyword-Driven Framework**
   → Rarely used nowadays, harder to maintain

👉 Don't answer casually. Your choice here impacts:

- Maintainability
- Scalability
- CI/CD integration
- Team collaboration

Answer with one option + reason.
Then I'll move to Question 2.

Hybrid Framework

Good. That's the only sensible choice if you're aiming for real-world scalability.

But just selecting "Hybrid" is meaningless unless you understand what you're combining.

Quick Reality Check

If your "Hybrid" = just POM + Excel → that's **basic**, not industry-grade.

A proper Hybrid Framework should include:

- POM (Page layer)
- Data-driven (external sources)
- Utilities (waits, drivers, configs)
- Reporting
- Logging
- Environment handling

- CI/CD readiness

If you're not planning all of this, you're building a **toy framework**, not an industrial one.

Question 2 (Execution Engine – Backbone)

Which test runner + build tool combination do you want?

Pick one:

1. **TestNG + Maven (Industry Standard – Strongly Recommended)**
2. **JUnit + Maven**
3. **TestNG + Gradle**
4. **Cucumber + TestNG + Maven (BDD Hybrid)**

👉 Be careful:

- If you pick Cucumber without real need → unnecessary complexity
- If you skip build tools → your framework is already broken

Answer with one option + reason.

TestNG + Maven

Correct choice. This is the backbone of most stable enterprise frameworks.

But don't assume this alone makes your framework "industrial."
If you misuse TestNG (no proper suite design, no parallel control, no listeners), it becomes messy fast.

Question 3 (Design Pattern – Maintainability Core)

How do you want to design your Page Layer?

Pick one:

1. **Classic Page Object Model (POM)**
   → One class per page, locators + actions together
2. **Page Factory (with @FindBy)**
   → Cleaner syntax, but often misused and less flexible
3. **POM + Separate Action Layer (Recommended – Advanced)**
   → Page = locators only
   → Actions = reusable methods (click, type, etc.)
4. **POM + Fluent Interface (Advanced)**
   → Method chaining (e.g., loginPage.enterUser().enterPass().clickLogin())

Reality Check

- If you pick only basic POM → your framework will grow messy
- If you over-engineer (Fluent everywhere) → becomes hard to debug
- Industry standard in mature teams = **POM + reusable action/util layer**

👉 **Answer with one option + reason.**
Then we move to Question 4.

POM + Fluent Interface (Advanced)

Alright — but this is where I push back.

Brutal Reality Check

POM + Fluent Interface sounds "advanced", but most teams misuse it.

Problems you'll hit if you're not careful:

- Debugging becomes painful (chained calls hide failures)
- Logs become unclear
- Reusability suffers if chaining is forced everywhere
- New team members struggle to maintain it

👉 Fluent Interface is useful **only for high-level business flows**, not for every method.

What mature teams actually do:

- POM + Utility/Action Layer (core)
- Fluent Interface → used selectively (login, checkout flows)

If you blindly apply Fluent everywhere → you're building a fragile framework.

I'll allow this choice, but only if you apply it **strategically, not blindly**.

Question 4 (Execution Strategy – Scaling & Performance)

How do you want to handle test execution?

Pick one:

1. **Sequential Execution (Basic – Not acceptable for real projects)**
2. **Parallel Execution using TestNG (Recommended)**
3. **Parallel + ThreadLocal WebDriver (Industry Standard)**
4. **Distributed Execution (Grid / Cloud + Parallel)**

Reality Check

- If your framework doesn't support parallel → it's already outdated
- If you use parallel without ThreadLocal → expect random failures
- If you plan CI/CD → parallel is **mandatory**

👉 **Answer with one option + reason.**
Then we move to Question 5.

Parallel + ThreadLocal WebDriver

Good. That's the correct engineering choice.

Reality Check (Important)

Most people *say* ThreadLocal, but implement it wrong:

- Static WebDriver → ❌ breaks parallel
- Shared instances → ❌ flaky tests
- Improper cleanup → ❌ memory leaks

If you don't manage:

- `ThreadLocal<WebDriver>`
- Proper `set()` and `get()`
- `remove()` after execution

→ Your framework will become unstable under load.

Question 5 (Test Data Strategy – Scalability Bottleneck)

How do you want to manage test data?

Pick one:

1. **Hardcoded Data (Unacceptable)**
2. **Excel (Apache POI)**
3. **JSON / YAML (Recommended for modern frameworks)**
4. **Database (Advanced – enterprise-level)**
5. **Combination (Excel + JSON + DB)**

Reality Check

- Excel is overused and slows execution
- JSON is cleaner, version-control friendly
- DB is powerful but overkill unless truly needed
- Hybrid approach is best *only if managed properly*

👉 **Answer with one option + reason.**
Then final Question 6.

Excel

That's a **practical but slightly outdated choice**—acceptable, but not optimal.

Reality Check

Excel works, but comes with baggage:

- Slower read operations (Apache POI overhead)
- Hard to maintain at scale
- Poor version control visibility (compared to JSON)
- Becomes messy with large datasets

👉 Still widely used in enterprise projects, so **acceptable if structured properly**:

- Separate sheets per module
- Avoid excessive columns
- Build a proper utility layer (don't read Excel in test classes directly)

Question 6 (Final – Reporting & Observability)

How do you want to handle reporting and logging?

Pick one:

1. **Extent Reports only**
2. **Log4j only**
3. **Extent Reports + Log4j (Industry Standard – Recommended)**
4. **Allure Reports + Logging (Modern alternative)**

Reality Check

- Reporting without logging → useless for debugging
- Logging without reporting → useless for stakeholders
- If your reports don't include screenshots → poor framework

👉 **Answer with one option + reason.**

After this, I will generate your **complete industrial-grade Selenium framework structure + design + best practices**.

Extent Reports + Log4j

Good. That's the correct combination for both **visibility (reports)** and **debugging (logs)**.

✅ Final Framework Design (Industrial Standard)

🔷 1. Framework Architecture (Hybrid)

Your framework will include:

```
src/main/java
 ├── base
 │    └── BaseTest.java
 │    └── DriverManager.java (ThreadLocal)
 │
 ├── pages
 │    └── LoginPage.java
 │    └── DashboardPage.java
 │
 ├── utils
 │    └── ExcelUtil.java
 │    └── ConfigReader.java
 │    └── WaitUtil.java
 │    └── ScreenshotUtil.java
 │
 ├── actions (Fluent Interface - controlled usage)
 │    └── LoginActions.java
 │
 ├── listeners
 │    └── TestListener.java (Extent Reports)
 │
 ├── reports
 │    └── ExtentManager.java
 │
 ├── constants
 │    └── FrameworkConstants.java
 │
 └── enums (optional but recommended)
      └── WaitStrategy.java


src/test/java
 ├── tests
 │    └── LoginTest.java
 │
 ├── testdata
```

```
│       └── TestData.xlsx
│
└── runners
        └── TestNG.xml
```

🔷 2. Core Design Decisions (What you chose + My correction)

Area	Your Choice	Verdict
Framework Type	Hybrid	✅ Correct
Runner	TestNG + Maven	✅ Industry standard
Design Pattern	POM + Fluent	⚠️ Use Fluent **only for flows**
Execution	Parallel + ThreadLocal	✅ Correct
Test Data	Excel	⚠️ Acceptable but limited
Reporting	Extent + Log4j	✅ Correct

🔷 3. Key Components (No shortcuts here)

✅ DriverManager (ThreadLocal – MUST be correct)

```
public class DriverManager {

    private static ThreadLocal<WebDriver> driver = new ThreadLocal<>();

    public static WebDriver getDriver() {
        return driver.get();
    }

    public static void setDriver(WebDriver driverInstance) {
        driver.set(driverInstance);
    }

    public static void unload() {
        driver.remove();
    }
}
```

✅ BaseTest (Setup + Teardown)

```
@BeforeMethod
@Parameters("browser")
public void setup(String browser) {
    WebDriver driver = BrowserFactory.initDriver(browser);
    DriverManager.setDriver(driver);
}
```

```
@AfterMethod
public void tearDown() {
    DriverManager.getDriver().quit();
    DriverManager.unload();
}
```

✅ Page Layer (Clean – No business logic)

```
public class LoginPage {

    private By username = By.id("user");
    private By password = By.id("pass");
    private By loginBtn = By.id("login");

    public void enterUsername(String user) {
        DriverManager.getDriver().findElement(username).sendKeys(user);
    }

    public void enterPassword(String pass) {
        DriverManager.getDriver().findElement(password).sendKeys(pass);
    }

    public void clickLogin() {
        DriverManager.getDriver().findElement(loginBtn).click();
    }
}
```

⚠️ Fluent Action Layer (Use smartly)

```
public class LoginActions {

    LoginPage loginPage = new LoginPage();

    public LoginActions login(String user, String pass) {
        loginPage.enterUsername(user);
        loginPage.enterPassword(pass);
        loginPage.clickLogin();
        return this;
    }
}
```

👉 Don’t over-chain everything. Keep flows readable.

✅ Excel Utility (Centralized – no duplication)

```
public class ExcelUtil {

    public static String getData(String sheetName, int row, int col) {
        // Apache POI logic
        return value;
    }
}
```

✅ Wait Utility (Explicit Wait – Mandatory)

```
public class WaitUtil {

    public static WebElement waitForElement(By locator) {
        WebDriverWait wait = new WebDriverWait(
            DriverManager.getDriver(), Duration.ofSeconds(10)
        );
        return
wait.until(ExpectedConditions.visibilityOfElementLlocated(locator));
    }
}
```

✅ Extent Report + Listener

```
public class TestListener implements ITestListener {

    public void onTestFailure(ITestResult result) {
        ScreenshotUtil.capture();
        ExtentManager.getTest().fail(result.getThrowable());
    }
}
```

🔷 4. TestNG Parallel Execution

```
<suite name="Suite" parallel="tests" thread-count="3">
    <test name="Test1">
        <classes>
            <class name="tests.LoginTest"/>
        </classes>
    </test>
</suite>
```

🔷 5. Critical Best Practices (Most people fail here)

🚫 Common Mistakes You MUST avoid

- Using `Thread.sleep()` → unacceptable
- Writing WebDriver directly in test classes → bad design
- Mixing locators + business logic → poor maintainability
- Overusing Fluent Interface → debugging nightmare
- Reading Excel in test methods → performance issue

✅ What makes your framework “Industrial”

- Thread-safe execution
- Clear separation of layers
- Reusable utilities
- Proper reporting + logging
- Scalable structure

- CI/CD ready (next step for you)

🔷 Final Verdict (Straight Talk)

You made mostly solid choices, but:

- Excel choice limits long-term scalability
- Fluent Interface can backfire if overused

👉 Overall: **7.5/10 framework design**